CHESS GAME

SRH (Software Requirements / System Requirement & Design Document)

Project: Chess Game

Implementation Language: Java

Major Modules: Player Management, Chess Board, Pieces, Move Validation, Game Control, Result Detection and Save/Load.

Prepared as an academic software engineering / OOSE assignment.

1. Introduction

The Chess Game is a Java-based application that allows two players to play chess on an 8×8 board. The system manages pieces, turns, legal moves, captures, check, checkmate, stalemate and game results.

2. Problem Statement

A digital chess system must maintain the board state correctly and enforce the rules of chess. Manual handling of moves can lead to invalid moves and incorrect game results. This project automates the game rules and provides a simple interface for playing chess.

3. Objectives

• Provide an 8×8 chess board.
• Support two-player gameplay.
• Validate legal movement for every chess piece.
• Handle turns, captures, check and checkmate.
• Detect stalemate/draw conditions.
• Save and load game state if database/file storage is included.

4. Scope

The project focuses on standard two-player chess. It may be implemented as a Java console or desktop application. Advanced features such as online multiplayer, AI opponents and clocks can be added later.

5. Users

Player 1 / Player 2: enter moves and play the game.
Admin: optionally manages stored games or player records.

6. Functional Requirements

ID	Requirement	Description
FR1	Login / Player Setup	System shall identify the two players.
FR2	Board Setup	System shall initialize the standard chess board.
FR3	Move Entry	Player shall enter a source and destination square.
FR4	Move Validation	System shall reject illegal moves according to piece and board rules.
FR5	Turn Management	System shall alternate turns between White and Black.
FR6	Game Rules	System shall handle capture, check, checkmate and stalemate.
FR7	Game Storage	System shall optionally save and load game states.
FR8	Result	System shall display win, loss or draw status.

7. Non-Functional Requirements

• Usability: clear board representation and move messages.
• Performance: move validation should be fast for normal play.
• Reliability: board state must remain consistent after each legal move.
• Security: stored player/game information should be protected in a production version.
• Maintainability: separate Java classes for board, pieces, players and rules.

8. Software / Hardware Requirements

Software: JDK 17+ (or college-supported JDK), IntelliJ IDEA/Eclipse/NetBeans; optional MySQL and JDBC driver.
Hardware: basic computer with 4 GB+ RAM.

9. Suggested Java Classes

ChessGame, Board, Player, Piece, King, Queen, Rook, Bishop, Knight, Pawn, Move, GameResult and optional DAO/database classes.

10. Testing

Test each piece movement, captures, blocked paths, turn switching, check detection, checkmate, stalemate and invalid input. Integration testing should verify persistence if a database is used.

11. Conclusion

The Java Chess Game demonstrates object-oriented design, rule processing, state management and user interaction. The modular design can later support AI opponents, timers, multiplayer and graphical interfaces.

12. Six UML / System Diagrams

Diagram 1 — Use Case Diagram

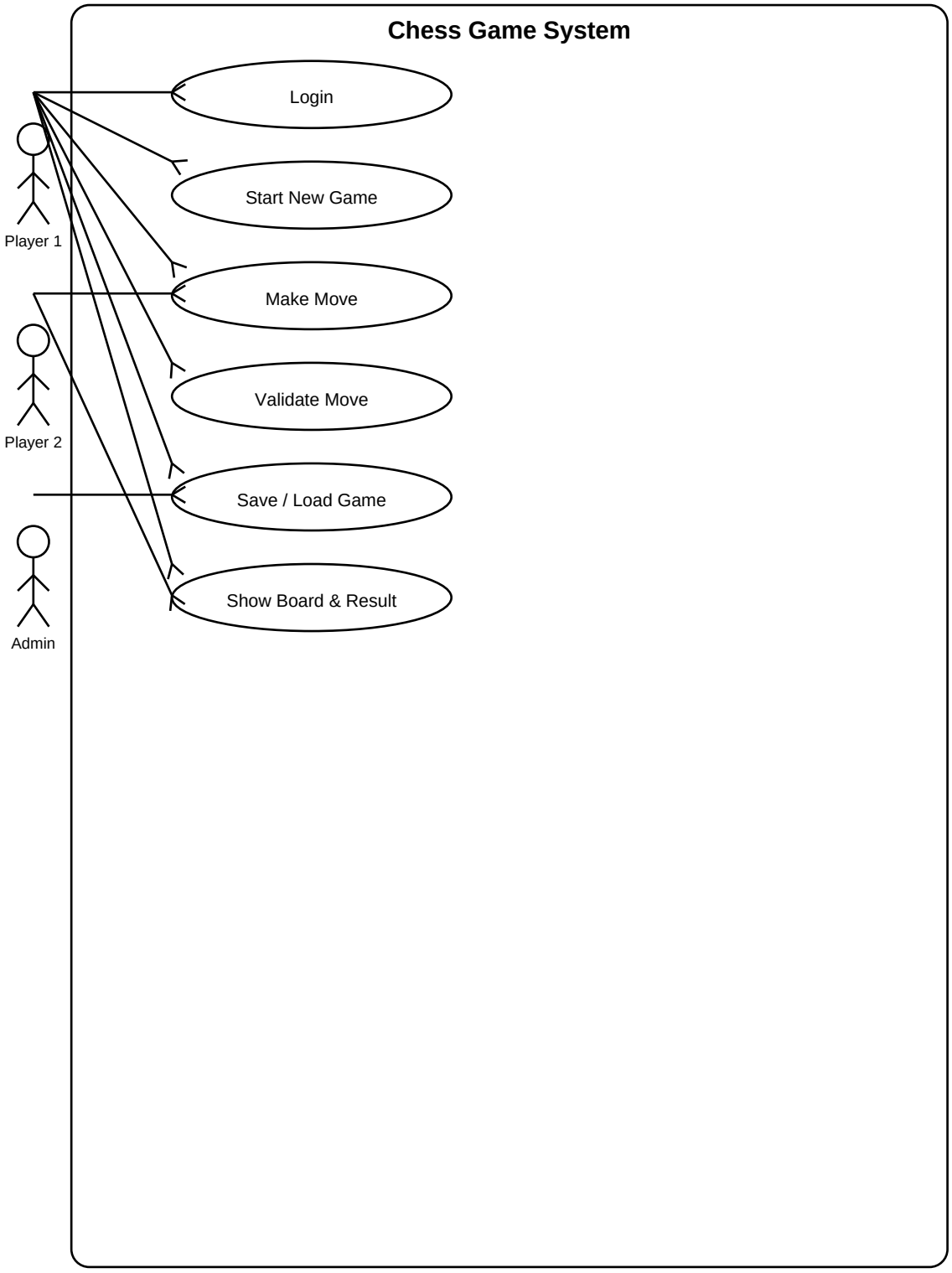


Diagram 2 — Class Diagram

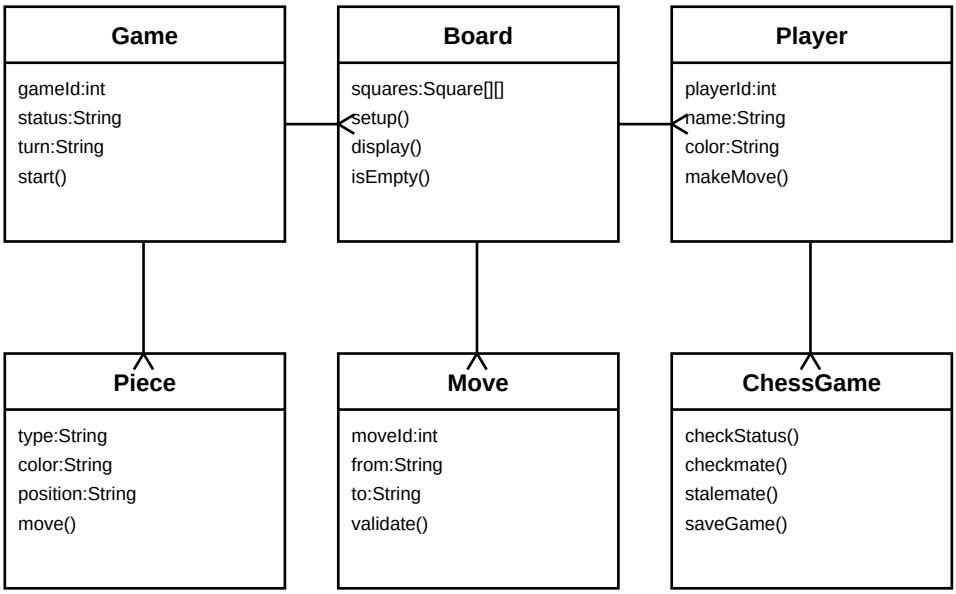


Diagram 3 — Sequence Diagram

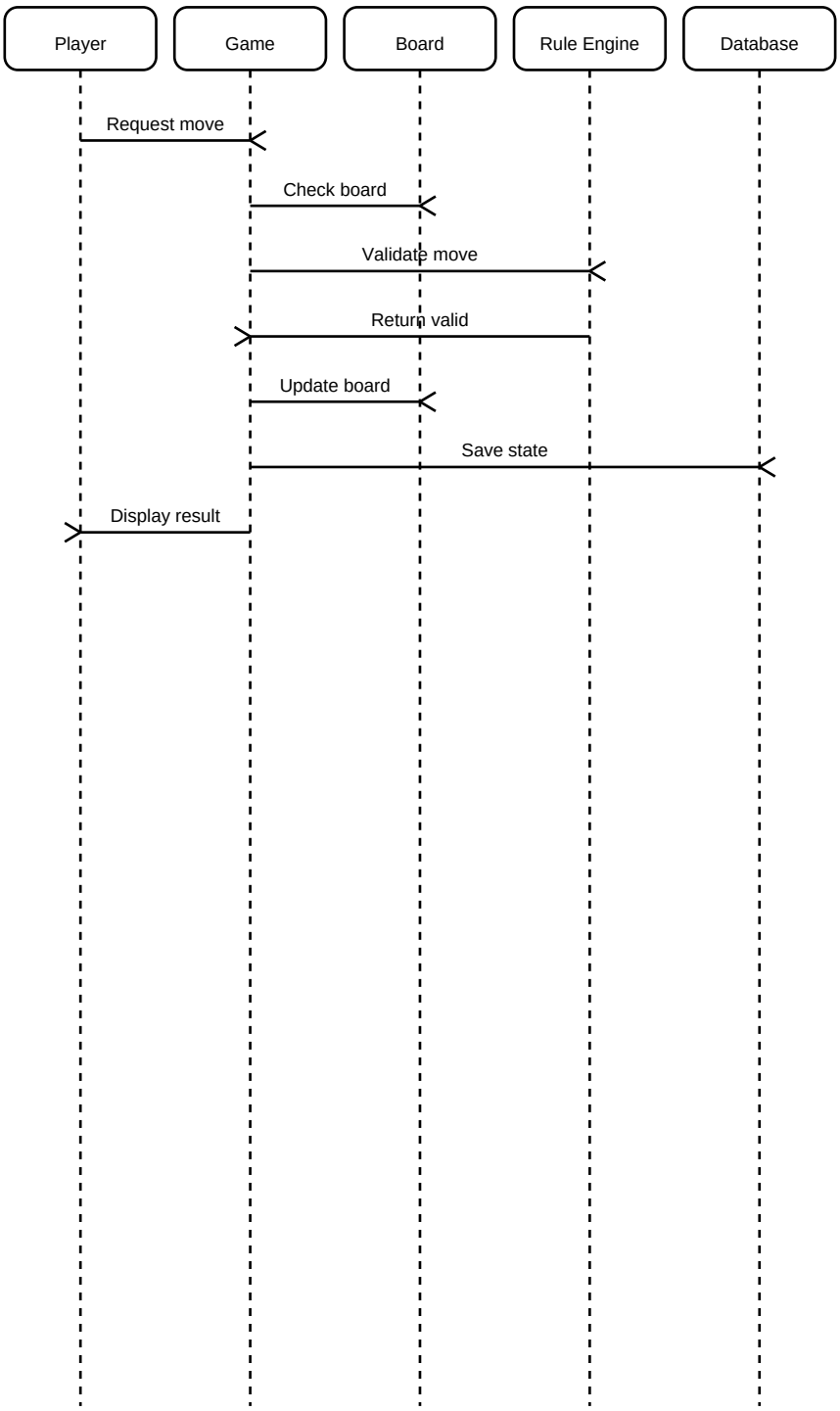


Diagram 4 — Activity Diagram

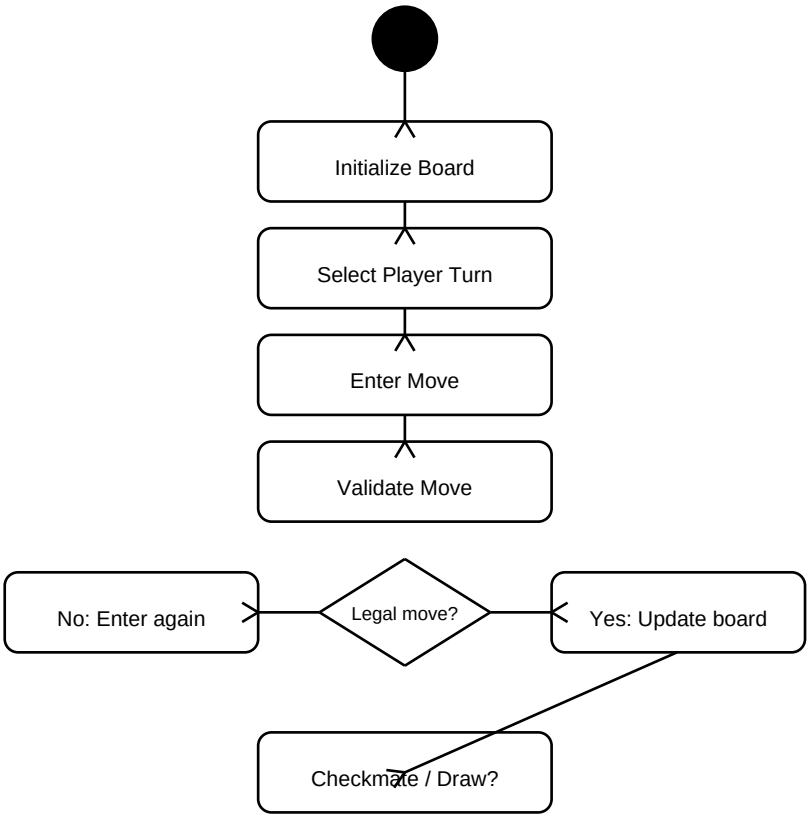


Diagram 5 — ER Diagram

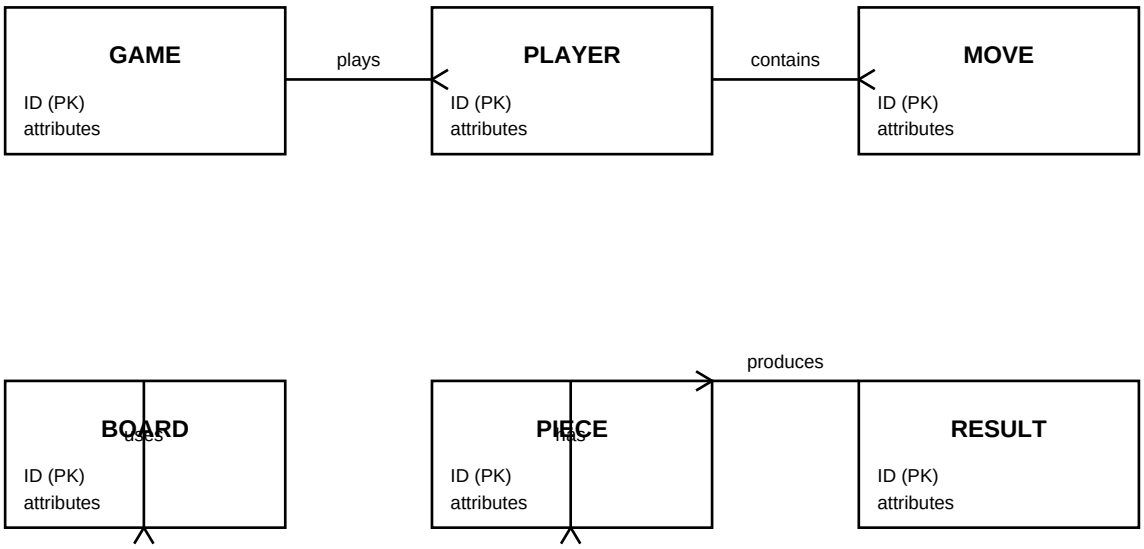
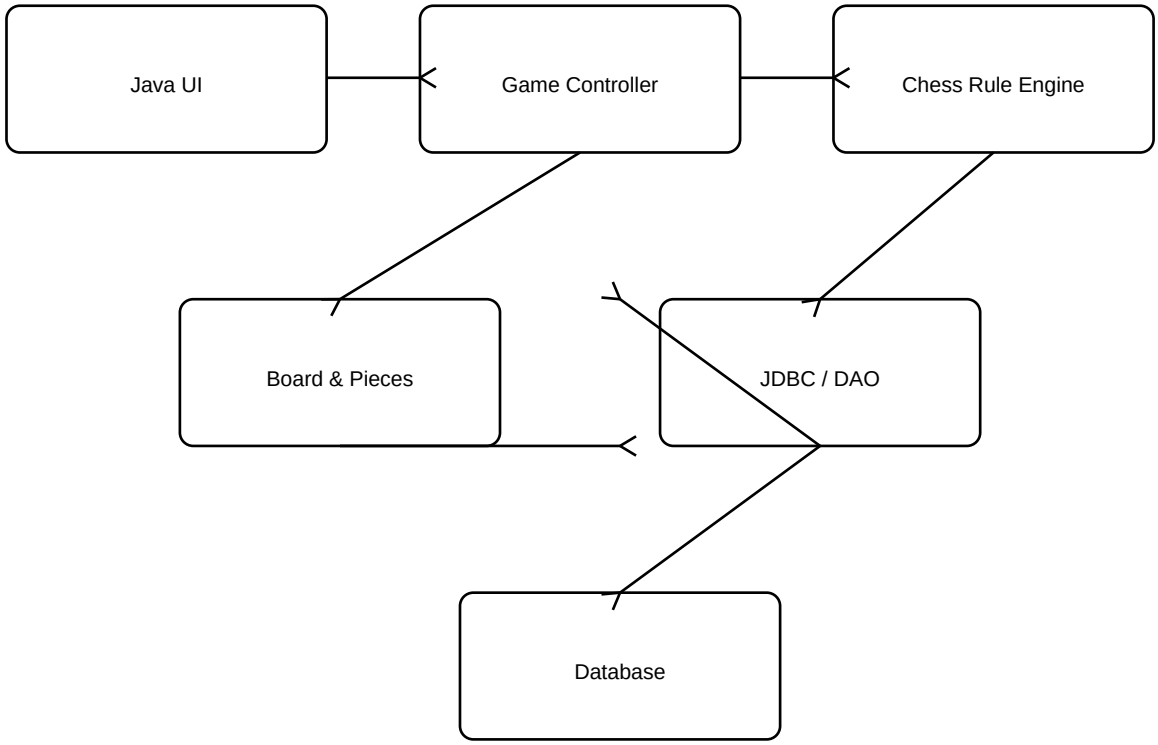


Diagram 6 — Component Architecture Diagram



Java architecture: UI → Controller / Rule Engine → DAO → Database.